# Problem A. A Challenge For a Tourist

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 3 seconds |
| Memory limit: | 512 mebibytes |

A tourist is completing challenges. There are $n$ cities, numbered from 1 to $n$, and there are $m$ bidirectional roads between some pairs of cities. The $i$-th road connects cities $u_i$ and $v_i$ and has a *difficulty* $w_i$, which is a nonnegative integer.

A *path* between cities $f$ and $t$ is a pair $(C, R)$, where

- $C = (c_0, \ldots, c_k)$ is a sequence of cities, where $c_0 = f$ and $c_k = t$,
- $R = (r_1, \ldots, r_k)$ is a sequence of roads, and for all $i$ from 1 to $k$ the road $r_i$ connects cities $c_{i-1}$ and $c_i$.

Note that a path may contain the same city more than once, and the same goes about the roads.

If $r$ is a road, denote by $w(r)$ its difficulty. For a path $(C, R)$, let its *difficulty* be $\max\{w(r) \mid r \in R\}$, and let its *hash* equal $w(r_1) \oplus \cdots \oplus w(r_k)$. Here by $x \oplus y$ we denote the xor (exclusive or) of $x$ and $y$.

Finally, a *challenge* is a triple $(f, t, h)$ denoting that the tourist has to travel from the city $f$ to the city $t$ using a path with hash equal to $h$.

Of course, the tourist wants to spend as little effort for each challenge as possible. Given $q$ challenges, for each of them find the lowest difficulty of a path that fits into the constraints of the challenge, or report that there is no way to finish the challenge.

## Input

The first line contains two integers $n$ and $m$, separated by space ($1 \le n, m \le 200\,000$; $n \ge 2$).

The $i$-th of the following $m$ lines contains three integers $u_i$, $v_i$, $w_i$ ($1 \le u_i, v_i \le n$; $0 \le w_i < 2^{30}$) that represent the $i$-th road. There may be more than one road connecting the same pair of cities, and there may be roads that connect the same city with itself. The cities don't have to be accessible from each other, that is, there may be such pair of cities that there is no path between them.

The next line contains the only integer $q$, denoting the number of challenges ($1 \le q \le 200\,000$).

The $j$-th of the following $q$ lines contains three integers $f_j$, $t_j$, $h_j$ ($1 \le f_j, t_j \le n$; $0 \le h_j < 2^{30}$; $f_j \ne t_j$), representing the $j$-th challenge.

## Output

Print $q$ integers, $i$-th of them being the lowest possible difficulty of a path that completes the $i$-th challenge, or $-1$, if there is none.

## Example

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| 6 6<br>5 6 3<br>6 2 2<br>1 2 1<br>2 3 2<br>2 4 3<br>4 5 4<br>3<br>1 3 1<br>1 3 3<br>1 3 5 | -1<br>2<br>4 |

# Problem B. Brackets

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

While working on the very important part of the code, the programmer Dmitry used the autocorrecting tool to remove all extra brackets from the expression. Unfortunately, the tool accidentally removed all brackets from the line of code that Dmitry was not going to fix, and now the program cannot be compiled.

After Dmitry restored the original version, a new task came to his mind. Consider the following simplified version of a programming language:

- variables are single lowercase English letters: `var = ['a'-'z']`;
- pre-increment: `preincr = ++var`;
- post-increment: `postincr = var++ | preincr++`;
- operand: `operand = var | preincr | postincr | (operand)`;
- expression: `expr = operand | expr + expr`.

The character `'|'` is used to separate the options in the description above and **does not** appear in the expression.

Dmitry wants to know if there exists an algorithm that, for any such expression without the brackets, builds the expression with brackets in such a way that there is no ambiguity in parsing the expression. Specifically, the resulting expression must be valid, and for each `'+'` sign, we should be able to uniquely determine whether it is an addition sign, pre-increment or post-increment, and to which variable each increment is applied. Help him to find such a way to add brackets.

## Input

The first line of the input contains the non-empty string that consists of the letters between 'a' and 'z' inclusively and the '+' signs, such as there are no two consecutive letters (i.e. all the variable names are of length 1). The length of the string goes not exceed $10^6$.

You may assume that the string is a correct expression written in the programming language described in the statement.

## Output

In a single line print the expression that is built from the given one by adding some brackets, such that all the pre-increments and post-increments included in the expression are uniquely defined. The expression must be correct in terms of the programming language described in the statement.

The number of pairs of the brackets must not be greater than the number of the variables (i.e. the number of the English letters in the input).

If there are more than one solutions, print any of them.

## Examples

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| x+++y | (x++)+(y) |
| q+u+++h+++++o+q++ | (q)+(u++)+(h++)+(++o)+(q++) |

# Problem C. Credit Cards

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

The T-Bank is the first bank that issues credit cards in the shape of obtuse triangles for its clients.

Due to the bank's personal approach to the clients, the sizes of all sides of the triangles are **unique** integers in the range $[1, n]$, i.e. the configurator tool works in the following way.

Initially, the pool contains $n$ pairwise distinct integers between 1 and $n$. The configurator allows to choose any three integers from the current pool such that the triangle formed by three segments with the lengths equal to those integers is obtuse (i.e. one of its angles is strictly between 90 and 180 degrees), then the card is generated and those integers are **removed** from the current pool.

Find the maximum number of obtuse triangles that can be generated by the configurator for the given $n$.

## Input

The first line contains the only integer $n$ ($1 \le n \le 10^6$) — the number of available distinct integers in the configurator.

## Output

On the first line print a single integer $t$ ($0 \le 3 \cdot t \le n$) — the maximum number of obtuse triangles.

The $i$-th line of next $t$ lines should contain 3 integers separated with spaces $a_i$, $b_i$, $c_i$ ($1 \le a_i, b_i, c_i \le n$) — the side lengths of the $i$-th triangle.

If there are several possible solutions, print any of them.

## Examples

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| 3 | 0 |
| 4 | 1<br>2 3 4 |
| 9 | 2<br>2 3 4<br>6 5 9 |

# Problem D. Doors of the Wardrobe

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 3 seconds |
| Memory limit: | 512 mebibytes |

*There is a picture in the notes, we recommend looking at it often during the reading of the statement.*

Sergey bought a new wardrobe. In this wardrobe, the doors are installed in $N$ layers: if we look at the wardrobe, we can see the first door in front of us, the second is hidden behind it, and so on, up to the very last $N$-th door.

Sergey is a mathematician, so it's easier for him to think about these doors in such a way as if they lie on the same Euclidean plane.

Each door consists of a pair of sliding parts. Consider a door, we can assume that the door parts are the two half-planes separated by some straight line $L$. Note that the wardrobe is designed in a way that the line $L$ is **not necessarily** vertical.

Moreover, at some point on the line $L$ the two handles are placed, one handle for each part of the door. Sergey can grab a handle and pull it in the direction perpendicular to $L$, thus moving the corresponding half of the door in that direction. Sergey can pull two handles on the parts of the same door independently of each other.

To open the wardrobe, Sergey opens all the doors in turn from the 1-st to the $N$-th. After Sergey begins to open the next door, the sliding parts of all the previous doors can no longer be moved. Sergey can only pull a handle if it is not obstructed by the previous doors during the entire moving process. So if Sergey does not open some door wide enough, it may block the opening of some of the following doors in the future.

Sergey wants to get $M$ small items from his closet. Each item can be represented as a point of the plane with the given coordinates. He wants to open the doors so that none of the objects is blocked by any door.

As a mathematician, Sergey wants to minimize the total distance he has to move the handles. Help him calculate this minimum.

## Input

The first line of the input contains two integers, $N$ and $M$ — the number of the doors and the number of the items, respectively ($1 \le N \le 10^5$, $3 \le M \le 10^5$).

Then $N$ lines follow, describing the doors in the order from the first (the door on the surface) to the $N$-th (the deepest door inside). Each door is described by four real numbers, given with no more than 15 digits after the decimal point. Those numbers are the coordinates $x_h$ and $y_h$ of the point $H$ where both handles are located when the door is closed, and the coordinates $x_d$ and $y_d$ of the vector $D$, such that the straight line $L$ splitting the door into halves contains the point $H$ and is orthogonal to the vector $D$.

Then $M$ lines follow, containing the descriptions of the items. Each item is described by its coordinates $x$ and $y$ — two real numbers, given with no more than 15 digits after the decimal point.

The coordinates do not exceed 10 by the absolute value. The length of vector $D$ for each door is not less than $\frac{1}{10}$. You may assume that there are three items that are not collinear.

Note that the tests are generated in such a way that there are no degenerate cases (like all points are too close to be collinear etc), and thus the tests are not intended to create troubles with the numerical stability.

## Output

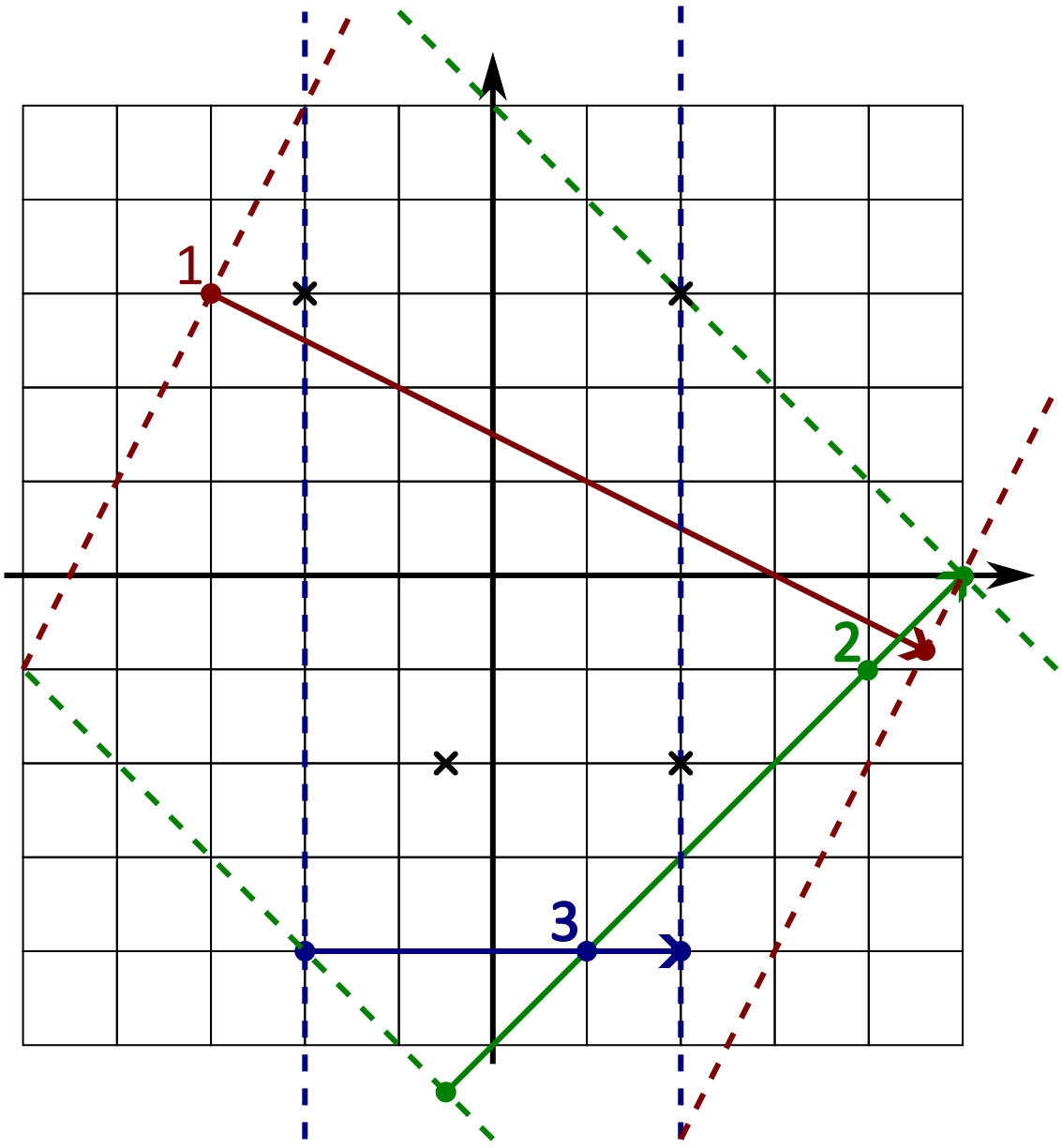
Print the minimal possible sum of the lengths of the handles movement vectors. The answer will be considered correct if the absolute or relative error from the optimal answer does not exceed $10^{-10}$.

## Example

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| 3 4<br>-3 3 0.6 -0.3<br>4 -1 1 1<br>1 -4 1 0<br>-2 3<br>2 3<br>2 -2<br>-0.5 -2 | 20.27523290755122432 |

## Note

There are three doors in the example, doors are marked with the red, green and blue colors. For each door the solid line denotes the trajectory of the handles movement, the dotted line denotes the positions of the halves of the door when they are moved. The black crosses denote the items.



# Problem E. Effective Management

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

Ilona is the CEO of a big industrial company. The company is manufacturing various kinds of products. For each type of product, Ilona knows two parameters: the profit from a sale of one item of that kind, and the *pure production time* for one item of that kind. The production of some products may depend on others. For example, to make a bicycle, you need to make a frame and wheels; to make wheels, you need spokes and tires; for a frame and spokes, you need processed metal.

Ilona plans to accelerate the production conveyor and produce all products in parallel in the required quantities. In her model, the *actual production time* of the product is equal to the **maximum** of the product's own pure production time and the pure production times of all its dependencies (not only direct ones). In the case of bicycle, its actual production time would be the maximum of the pure production times of the bicycle, frame, wheels, spokes, tires, and processed metal.

The company strives to earn as much as possible, so Ilona asks you to list all the products with the maximal possible ratio of the profit from sales to the actual production time.

## Input

The first line of the input contains one integer $n$ ($1 \le n \le 10^5$) — the number of the product types. The types are enumerated by sequential integers between 1 and $n$, inclusively.

The second and the third lines contain $n$ positive integers each. $i$-th integer in the second line is the profit from sale of the product of $i$-th type, and in the third line — the pure production time of the product of $i$-th type. Those integers do not exceed $10^9$.

Then $n$ lines follow, describing the production dependencies. $i$-th of those lines starts with the integer $m_i$ ($0 \le m_i < n$) — the number of types of products that the production of the product of $i$-th type depends on. Then $m_i$ integers follow, describing those types. You may assume that the sum of all $m_i$ does not exceed $10^5$.

You may assume that there are no cyclic dependencies in the production process, i.e. no type can depend on itself directly or indirectly.

## Output

Print the indices of the production types that maximize the ratio of the profit from sales to the actual production time, one index per line. The types shall be printed in the increasing order.

## Example

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| 6<br>1 2 4 5 4 6<br>1 3 2 2 3 2<br>0<br>0<br>1 1<br>2 2 3<br>1 1<br>2 5 4 | 3<br>6 |

# Problem F. Form processing

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

The IT department of Rosatom is now working on the unified platform for the domains of the various organizations. An important part of this platform is collecting and evaluating the client's view of the domain structure, interface, etc.

One of possible ways to collect this information is providing a special form that shall be filled by the client. To process the answers, the analysts of Rosatom develop special formula for each case.

The formula looks like

$$?s_1?s_2\ldots?s_n?$$

where $s_i$ is one of operators '+', '-', '*' and '/'. In other words, the formula can be represented as the string of odd length $2n+1$, where the '?' characters are placed on the positions with even indices $(0, 2, 4\ldots 2n)$, and operator signs are placed on the positions with odd indices $(1, 3\ldots 2n-1)$.

When processing the form with client's answer, each "?" is replaced by the client's answer — the digit between 1 and 9, inclusively.

When all "?" are replaced with the digits, the formula is evaluated using **rational arithmetics** (i.e. 2/6 = 1/3, and not 0 unlike the integer division in the contemporary programming languages nor 0.333...3 with a rounding error like the double division in those languages) and usual order of evaluation (i.e. the '*' and '/' have higher priority than '+' and '-').

When the value the formula evaluates to is **integer** (i.e. with zero fractional part), the client's answers are considered as consistent ones. The resulting integer is, in this case, the project value. Otherwise (if the fractional part is not zero) the answers are considered inconsistent, thus additional work with the client is needed and the project value is currently undefined.

You are assigned to write the part of the test suite, that, given the input formula, finds one possible sequence of "?" values that produces the maximal possible project value.

## Input

The input contains one string of an odd length, consisting of the characters '?', '-', '+', '*' and '/'. The even positions (considering that the positions are enumerated from zero) are used for '?', while the odd positions are filled with the operation signs. The length of the string does not exceed $2 \cdot 10^4 + 1$.

## Output

Print the string of the same length, where the ?'s are replaced with the digits from 1 to 9, such that the value of expression is integer and maximal possible.

If there are several possible solutions, print any of them.

## Examples

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| ?*?+?/?-? | 9*9+9/1-1 |
| ? | 9 |

# Problem G. Go Play Wall-nut Bowling!

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 10 seconds |
| Memory limit: | 512 mebibytes |



The famous computer game "Plants vs. Zombies" has a Wall-nut Bowling mode.

In this mode, the zombies move from right to left, and the player can launch wall-nuts at them, which bounce off the zombies on impact. Your task is to simulate the Wall-nut bowling game.

Please note that the rules used in this problem are **different** from the rules of the original game!

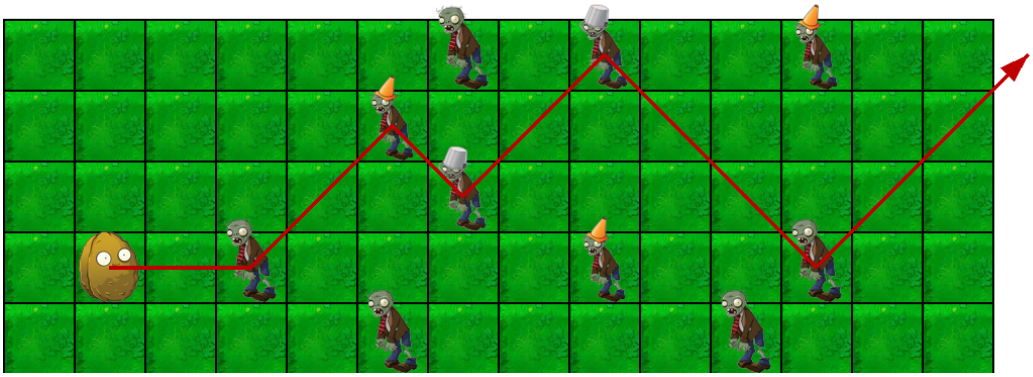
The game takes place on a playfield of size $M \times N$ unit cells. At any given time, there may be zombies in some cells of the field, with no more than one zombie in each cell.

The player can launch the wall-nut from any cell of the field. Immediately after launch, the nut rolls horizontally to the right until the first encounter with the zombies.

After each collision with a zombie, the nut rebounds and continues to move diagonally: either in the right-up or in the right-down direction.

If this collision is not the first, then the wall-nut continues to move along a different diagonal: if it rolled from the bottom-left, then it bounces down-right, and vice versa. If this is the first collision, then one of the two diagonals is chosen depending on how the player launched the nut.

The movement of the nut ends when it rolls out of the field to the right, down or up.



Thus, the nut draws a zigzag on the field. If the player launches a nut from the cell that has a zombie in it, then the nut immediately hits that zombie and bounces off of it. In this problem, we will assume that the nut is rolling very fast, and the start of the nut takes zero time.

All zombies move from right to left at unit speed, moving one square every second. When a zombie leaves the field, it enters your house and queues up to eat your brains, and does not participate in the game anymore.

In addition, new zombies appear in the game from time to time.

The only way to protect yourself from zombies is to beat them with wall-nuts!

Each zombie has an armour level. Each time a wall-nut hits a zombie, its armour level is reduced by one.

If, due to the impact, the armour level is reduced to zero, then this zombie collapses, i.e. completely disappears from the game.

The initial field is empty.

Your task is to perform $Q$ queries of the following three types:

- Wait $t$ seconds of time, letting the zombie to move to the left and/or enter the house.
- Launch a wall-nut from a given cell, hitting and possibly destroying some zombies.
- Add a new zombie in the given cell.

Launching a wall-nut and adding zombies is done in a negligible amount of time, however, all requests must be performed strictly in the given sequence.

## Input

The first line of the input contains three integers: $M$ is the number of the rows, $N$ is the number of columns, and $Q$ is the number of the queries ($1 \le M, N \le 10^7$, $1 \le Q \le 3 \cdot 10^5$).

The following $Q$ lines contain the queries, in order, one query per line. Each query consists of the command description, followed by some integers.

**time t** — wait $t$ seconds ($1 \le t \le 10^7$). You shall print one integer — the number of zombies that left the playfield and joined the queue to eat your brains for that time interval.

**deliver r c s** — shoot the nut. Here $r$ and $c$ — the row and the column of the starting cell for the nut ($1 \le r \le M$, $1 \le c \le N$), and $s$ determines the direction of bounce: $-1$ for right-up direction and 1 for right-down direction. You shall print two integers — the number of bounces and the number of zombies destroyed by this shoot.

**add r c h** — add the zombie. Here $r$ and $c$ are the row and the column of the cell where a zombie is added ($1 \le r \le M$, $1 \le c \le N$), and $h$ is the initial level of the armour ($1 \le h \le 10^6$).

When the nut moves down, the row index is increasing, when the nut moves right, the column index is increasing.

You may assume that a zombie will never be added to the cell with another zombie. You may assume that the sum of $t$ in the `time` queries does not exceed $10^9$.

## Output

In the first line of the output print the text `stdout ducks` because the zombies in this game use the ducks.



Then print the answers on the `time` and `deliver` queries, one answer per a line.

## Example

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| 6 8 23<br>add 2 4 2<br>add 3 7 1<br>add 4 6 3<br>add 5 3 3<br>add 6 4 3<br>deliver 1 1 1<br>deliver 5 1 1<br>deliver 5 1 -1<br>deliver 2 4 1<br>deliver 5 1 1<br>time 2<br>add 4 4 3<br>deliver 2 1 1<br>deliver 6 1 -1<br>time 7<br>add 2 3 1<br>add 2 6 1<br>add 3 3 1<br>add 4 4 1<br>deliver 2 5 1<br>add 3 7 10<br>deliver 3 2 1<br>time 5 | stdout ducks<br>0 0<br>3 0<br>1 0<br>3 1<br>3 2<br>0<br>2 1<br>2 1<br>1<br>1 1<br>2 2<br>1 |

# Problem H. Hacking the Project

| | |
|---|---|
| Input file: | *standard input* |
| Output file: | *standard output* |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

*This is an interactive problem.*

Vasya is one of the developers of the new programming language called DiverC. The main feature of the program written in this language is that the code consists of pairwise distinct words. The compiler of DiverC developed by Vasya is, of course, written in DiverC and consists of $N$ pairwise distinct words.

Vasya is using the DiverC online autofill service. But Vasya has made one serious mistake: he forgot to switch the "use my data for the improvement of the service database" function off. And Vasya was the first person who registered on this service, so now the service contains only the words from his compiler.

Hacker Boris wants to know all the words Vasya used in the compiler. So he registered at the DiverC online autofill service (wisely switching the dangerous function off), and now, for each prefix $S$ and integer $K$ entered by Boris, the service returns, in lexicographic order, the first $K$ words from Vasya's code that begin with the prefix $S$. If there are only $k < K$ words, the service gives out only $k$ words (but the service usage counter increases by $K$ even in this case).

Boris checked the scripts used for the online service and found that one user is limited with 3 800 as the total value of $K$ in all queries. He wants to determine all $N$ words used by Vasya with several queries such as the sum of $K$ in those queries is 3 800 or less.

Can you help him?

## Interaction Protocol

In the beginning, your program shall read one integer $T$ — the number of the test cases to be processed ($1 \le T \le 5$).

At the beginning of each test case, the jury program tells one integer $N$ — the number of the words in Vasya's DiverC compiler ($1 \le N \le 1\,000$).

Your program can then make two types of requests:

- `query S K` — get $K$ ($1 \le K \le N$) lexicographically minimal words starting with prefix $S$ ($1 \le |S| \le 10$). If the dictionary contains only $k$ such words, where $k < K$, the answer to the query will contain $k$ words. The response to the query will be one line of the form $kS_1S_2 \ldots S_k$, where $k$ is the number of the words ($0 \le k \le K$), and then $k$ words $S_i$ in lexicographic order follow.
- `answer` $S_1$ $S_2$ $\ldots S_N$ — tell the full Vasya's dictionary. After the word `answer` you shall print all $N$ words in an arbitrary order separated by spaces. There will be no response from the jury program to this request, and your program must then continue with the next test case or exit if the current test case was the last one.

The words in Vasya's code are composed of lowercase English letters. The length of words is between 1 to 10 characters. All words in Vasya's code are pairwise distinct.

The sum of $K$ for all queries of the first type for each test should be 3 800 or less.

Violating the interaction protocol or exceeding the limits for the sum of $K$ cause the "`Wrong answer`" verdict.

Make sure you print the newline character after each query and flush the output stream buffer (`flush` language command) after each request. Otherwise, the solution may get the idleness limit exceeded verdict.

Note that the jury program is **adaptive**, i.e. the set of Vasya's words may be generated at the runtime, but the set is guaranteed to be consistent with the answers to previous queries.

## Example

| *standard input* | *standard output* |
|---|---|
| 1<br>4<br><br>1 aaa<br><br>2 aaa aba<br><br>1 cxyxy<br><br>0<br><br>1 czzzz | <br><br>query a 1<br><br>query a 4<br><br>query c 1<br><br>query cy 1<br><br>query cz 1<br><br>answer aaa aba czzzz cxyxy |

# Problem I. Inspection (32 MB ML!)

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 5 seconds |
| Memory limit: | 32 mebibytes |

**Note the unusual memory limit on this task!**

After the road inspection considered the high frequency of the potholes on each kilometer of the road, the regional government assigned some money to carry out partial road repairs.

Using the given money, it is possible to repair $K_1$ continuous road segments of length exactly $L_1$ kilometers, as well as $K_2$ continuous segments of length exactly $L_2$ kilometers. As a result of the roadworks, all the potholes on each of the repaired sections of the road are eliminated.

You were hired by the road building company to choose which sections of the road need to be repaired, to eliminate as many potholes as possible. Unluckily, due to budget restrictions, the PC that is provided to you for this task has **only 32 mebibytes** of free memory for your program.

The bureaucratic rules demand the repaired sections start (and therefore end) at an integer distance from the beginning of the road of kilometers.

The repaired sections cannot overlap with each other. All the money allocated for the repair of the road must be used.

## Input

The first line of the input contains five integers: $N$ is the length of the road ($1 \le N \le 10^4$), $K_1$, $L_1$ are the number of the sections of type 1 and the length of the section of type 1, respectively, $K_2$, $L_2$ are the number of the sections of type 2 and the length of the section of type 2, respectively ($1 \le K_i, L_i \le 100$, $L_1 \ne L_2$, $K_1L_1 + K_2L_2 \le N$).

The second line contains $N$ integers, $i$-th of those integers is the number of the potholes at the $i$-th kilometer of the road. Those integers are non-negative and do not exceed $10^9$.

## Output

In the first line of the output print one integer — the total number of the potholes to be eliminated after the roadworks.

In the following $K_1 + K_2$ lines print two integers: the coordinates of the beginning and the end of the section to be repaired. The sections shall be sorted in increasing order of the coordinates.

The coordinate of the beginning of the road is 0.

## Example

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| 20 2 3 3 2<br>2 0 2 0 1 2 1 2 1 2 2 2 1 1 2 2 2 1 2 2 | 22<br>4 6<br>6 8<br>9 12<br>14 17<br>18 20 |

# Problem J. Jeopardized Betting

| | |
|---|---|
| Input file: | *standard input* |
| Output file: | *standard output* |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

*This is an interactive problem.*

In knockout competitions, two teams play matches between each other until one of the teams wins $N$ of them. At the same time, there can be no tied matches, so each match ends with the victory of one of the teams.

Your favourite team is playing in one of these competitions and you want to bet on this team winning the series. The bet should work like this: if your team wins the series, double the bet is returned, and if it loses, nothing is returned.

However, the bookmaker you are using wants you to place as many bets as possible. Instead of giving the opportunity to bet on the outcome of the entire series of games, you are offered only to bet on each game separately.

But you didn't give up and accepted the bookmaker's challenge. You want to bet on each match in such a way that by the end of the series the amount of money you have either doubles (if your team wins), or is spent on bets and you have nothing left (in case of defeat of your team).

## Interaction Protocol

First, your program must read an integer $N$ — the number of matches required to win the series ($1 \le N \le 30$).

Initially your money is equal to $2^{2 \cdot N}$.

Then your program must bet on the first game of the series. To bet, you shall print a line containing one integer $b$ — the amount of money used to bet. The value of $b$ shall be strictly positive and should not exceed the amount of money you currently have.

The betting process then proceeds as follows.

After placing a bet, the jury program tells you the result of the match:

- `Won` — means that your team won and you get double your bet.
- `Lost` — means that the bet did not play and the money will not be returned to you.

If after the next match one of the teams scores $N$ wins, then the series of games ends and your program should exit.

If the series continues, then you must bet on the next game.

It is guaranteed that there always exists a sequence of bets that will solve the task.

Make sure you print the newline character and flush the output stream buffer (`flush` language command) after each printed bet. Otherwise, the solution may get the idleness limit exceeded verdict.

## Example

| *standard input* | *standard output* |
|---|---|
| 3<br><br>Lost<br><br>Won<br><br>Won<br><br>Won | <br>40<br><br>20<br><br>40<br><br>44 |

# Problem K. KPI Calculation

| | |
|---|---|
| Input file: | *standard input* or input.txt |
| Output file: | *standard output* or output.txt |
| Time limit: | 1 second |
| Memory limit: | 512 mebibytes |

Vasya, who worked as security in an IT company, suddenly got the position of the project manager in this company.

After reading the articles on the Internet related to code style and indentation, Vasya invented the new KPI parameter for the programmers, called the *codestyle score*.

It is defined as the sum of the number of spaces between the beginning of the line and the first printable character (i.e. a character with ASCII code strictly greater than the code of space) for all lines that contain **at least one** printable character.

Vasya asks you to write the program that calculates the codestyle score of the given piece of code.

## Input

The input is a single text file consisting of characters with ASCII codes between 32 and 126 inclusively and the line feeds.

Note that there may be lines ending with several spaces, lines consisting entirely of the spaces, and empty lines (including the empty lines at the end of the file).

The size of the input does not exceed 100KiB.

## Output

Print one integer — the codestyle score of the given input.

## Example

| *standard input* or input.txt | *standard output* or output.txt |
|---|---|
| #include <cstdio><br><br>int main()<br>{<br> int a,b;<br>  scanf ("%d%d",&a,&b);<br>   printf ("%d\n",a+b);<br>    return 0;<br>} | 10 |

# Problem L. Limited Shuffle Restoring

| | |
|---|---|
| Input file: | *standard input* |
| Output file: | *standard output* |
| Time limit: | 3 seconds |
| Memory limit: | 512 mebibytes |

*This is an interactive problem.*

Zhenya had an array $a$, initially equal to $(1, 2, \ldots, n)$. He did the following operations with the array.

- For each $i$ from 1 to $n$ in this order, Zhenya picked some index $j$ such that $i \le j \le \min(n, i + 2)$, and swapped $a_i$ and $a_j$. Of course, if $i = j$, then nothing happened after the operation.

Your goal is to determine the final array. You may ask questions of the following type.

- `? i j` meaning the question "How do $a_i$ and $a_j$ compare to each other?". Zhenya will respond to this with one symbol `<` or `>`, meaning that $a_i < a_j$ or $a_i > a_j$, respectively.

You may ask no more than $\lfloor 5n/3 \rfloor + 5$ questions. After this, you must guess the array.

## Interaction Protocol

First, the interactor prints the number $n$ in a separate line ($1 \le n \le 30\,000$). Then the solution makes queries, where each query consists of printing `? i j` on a separate line, where $1 \le i, j \le n$, and $i \ne j$. After each query the interactor prints one character `<` or `>` on a separate line.

After the solution has finished asking questions, it must make a guess. If you think that the array is $(a_1, \ldots, a_n)$, print `!` $a_1$ $a_2$ $\ldots$ $a_n$ on a separate line and terminate.

If your solution makes more than $\lfloor 5n/3 \rfloor + 5$ queries, the interactor will finish with the WA verdict. If you do not flush the output after printing a query, you may receive the IL verdict.

Note that the interactor in this task is **adaptive**, i.e. the array may be generated at the runtime consistently with your questions.

## Example

| *standard input* | *standard output* |
|---|---|
| 5<br><br><<br><br>><br><br>><br><br><<br><br>><br><br>> | ? 5 4<br><br>? 5 1<br><br>? 5 3<br><br>? 3 1<br><br>? 2 1<br><br>? 5 2<br><br>! 2 3 1 5 4 |